VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Fall Semester 2025–2026

Digital System Design with FPGA

Lab-8, 9, 10 Report

Blues Piano Using Terasic DE2-115 Development Board

Course Code: MAVLD503

Slot: L25 + L26

Submitted to:

Dr. Jagannadha Naidu K

Submitted by:

Name	**Reg. No.**
Spandan Bhaduri	26MVD0062
Pulavarti Yogesh	26MVD0064
Chinmayananda R Sharma	26MVD0080

1. Objectives / Specifications

1.1 Objectives

The aim of this project is to design and implement a Blues Piano on the Terasic DE2-115 FPGA development board using Verilog HDL. The piano plays the notes of a D-flat blues scale and the objectives are:

- To design a synthesizable Verilog system that reads an eight-key push-button keyboard and plays the selected note as a square wave on a speaker.
- To resolve simultaneous key presses with a priority encoder so that only one note is played at a time.
- To generate the required note frequencies from the 50 MHz board clock using PLL-derived clocks and binary up-counters acting as frequency dividers.
- To display the note being played (note letter and flat sign) on the two on-board seven-segment displays.
- To verify the design through simulation with a Verilog testbench and then implement it on hardware through pin assignment of the DE2-115 GPIO header.

1.2 Specifications

Parameter	Specification
Development board	Terasic DE2-115 (Altera Cyclone IV E FPGA)
Hardware description language	Verilog HDL
System clock	50 MHz on-board clock (rawclk), multiplied/divided using PLLs
Inputs	Eight push-button keys (active-low), connected through the 40-pin GPIO header (GPIO 5, 7, 9, 11, 13, 15, 17, 19)
Audio output	Square wave on one GPIO pin, fed to a speaker through an audio jack
Display output	Two on-board seven-segment displays (note letter and flat sign)
Notes played	Eight keys covering the D-flat blues scale: D♭, E, G♭, G, A♭, B, and the upper-octave D♭ and E
Polyphony	Monophonic – one note at a time, highest-numbered key pressed has priority
Tools	Intel Quartus (synthesis, pin assignment) and ModelSim – Intel FPGA Starter Edition 2020.1 (simulation)

1.3 Key-to-Note Mapping

Key	Priority	Note	Display (HEX2 HEX1)
keys[7]	Highest	D♭	D ♭
keys[6]		E	E
keys[5]		G♭	G ♭
keys[4]		G	G
keys[3]		A♭	A ♭
keys[2]		B	b
keys[1]		D♭ (octave up)	D ♭
keys[0]	Lowest	E (octave up)	E

1.4 Components Required

- DE2-115 Terasic development board
- 40-pin breakout ribbon cable with header connector
- Breadboard
- Eight push-button switches (keyboard keys)
- Eight 2 kΩ resistors (or resistors of similar value)
- Eight 0.022 µF capacitors (optional)
- One or two 10 µF capacitors
- Audio jack and speaker

2. Architecture / Block or Sub-block Diagram

The DE2-115 board is connected to a breadboard carrying the push-button keyboard and the audio-jack circuit through a 40-pin ribbon cable. The eight keys are connected to the GPIO header, and the audio output is taken from another GPIO pin and passed to the speaker. Figure 1 shows the complete hardware setup.



Figure 1: Hardware setup – DE2-115 board, breadboard keyboard and speaker

The Verilog design is organised hierarchically. The top-level module (Main) calls the Keyboard module to find out which note, if any, should be played, and passes this information on to the Seven Segment Display module and to the Tone Maker module. The Tone Maker selects one of the divided clock signals produced by the Clock Scheme Array, which is driven by PLLs and up-counters. Figure 2 shows the block diagram of the design.

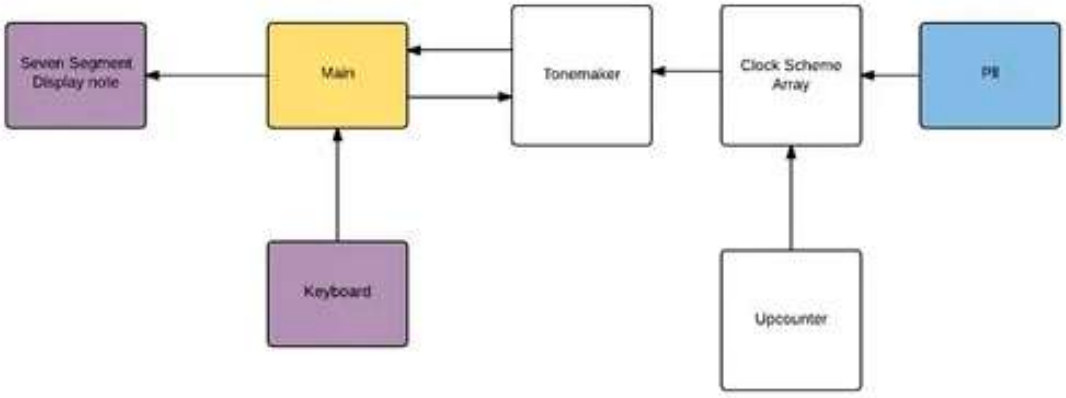


Figure 2: Block diagram of the Blues Piano Verilog modules

2.1 Module Description

Module	Function
Main (projectmaint)	Top-level module. Inverts the active-low key inputs and instantiates the keyboard, tonemaker and seven-segment modules, connecting the 4-bit note state between them.
Keyboard	Priority encoder that converts the eight key inputs into a 4-bit note code (0–7); code 15 means no key is pressed.

Module	Function
Tone Maker	Uses the note code to select one bit of the appropriate divided-clock counter and drives it to the speaker output as a square wave. Outputs 0 when no key is pressed.
Clock Scheme Array	Collects the PLL clock outputs and feeds each one to its own 32-bit up-counter, giving six independent frequency sources.
PLL	Generates the required higher-rate clocks from the 50 MHz input. Each on-board PLL can provide up to five output clock signals; the design uses two PLL instances (pll and pll2) created with the Quartus megafunction library.
Up-counter (upcounter32)	32-bit binary counter. Each counter bit toggles at half the rate of the previous bit, so tapping a particular bit gives a divided square-wave clock.
Seven Segment Display note	Decodes the note code into the seven-segment patterns for the note letter (HEX2) and the flat sign (HEX1).

2.2 Working Principle

Each note is produced by dividing a PLL clock with a 32-bit counter. The counter bit chosen in the Tone Maker sets the output frequency; selecting the next lower bit doubles the frequency, which is how the upper-octave notes are obtained from the same counters. The note-to-clock selection used in the design is given below.

Note code	Note	Speaker source	Key
0	D♭	clkarray0[15]	keys[7]
1	E	clkarray1[15]	keys[6]
2	G♭	clkarray2[15]	keys[5]
3	G	clkarray3[15]	keys[4]
4	A♭	clkarray4[15]	keys[3]
5	B	clkarray5[15]	keys[2]
6	D♭ (octave up)	clkarray0[14]	keys[1]
7	E (octave up)	clkarray1[14]	keys[0]
15	No key (off)	Speaker = 0	None pressed

Note: the Tone Maker also decodes code 8 (G♭ octave up, clkarray2[14]); this code is not generated by the keyboard module in this eight-key version.

3. Verilog Design Code

The design file (projectmaint.v) contains the top-level module and all sub-modules. The PLL modules (pll, pll2) are Quartus megafunction (IP) blocks generated in the Quartus project and are instantiated, not defined, in this file.

```
/* Main Control */
module projectmaint(
    input rawclk,       // 50MHz clock signal
```

```
    input [7:0] keys,   // Array of key inputs. Default HIGH.
    output speaker,     // Audio output (Square wave)
    output [6:0] hex1,  // Seven segment display output (Note letter)
    output [6:0] hex2   // Seven segment display output (Accidental)
);

    wire [3:0] statevalue;
    wire [7:0] gkeys;

    // Invert buttons so active-low becomes active-high
    assign gkeys = ~keys;

    // Module calls
    tonemaker tonemakerinst1(
        .rawclk(rawclk),
        .statevalue(statevalue),
        .speaker(speaker)
    );

    keyboard keyinstance1(
        .key(gkeys),
        .out(statevalue)
    );

    sevsegdisplaynote disnoteinstance1(
        .out1(hex2),
        .out2(hex1),
        .note(statevalue)
    );

endmodule

/* ############################################################################## */
/* Clock Scheme Array Module */
module clockschemearray(
    input rawclk,
    output [31:0] clkarray0,
    output [31:0] clkarray1,
    output [31:0] clkarray2,
    output [31:0] clkarray3,
    output [31:0] clkarray4,
    output [31:0] clkarray5
);
    wire slwclk0, slwclk1, slwclk2, slwclk3, slwclk4, slwclk5;
    wire plllocked;

    // Upcounters functioning as clock dividers
    upcounter32 counterinstance0(slwclk0, clkarray0);
    upcounter32 counterinstance1(slwclk1, clkarray1);
    upcounter32 counterinstance2(slwclk2, clkarray2);

    // Quartus PLL Megafunction Instantiations
    pll pllinstance1(0, rawclk, slwclk0, slwclk1, slwclk2, plllocked);

    upcounter32 counterinstance3(slwclk3, clkarray3);
    upcounter32 counterinstance4(slwclk4, clkarray4);
    upcounter32 counterinstance5(slwclk5, clkarray5);

    pll2 pllinstance2(rawclk, slwclk3, slwclk4, slwclk5);

endmodule

/* ############################################################################## */
/* Tone Generation Selection Module */
module tonemaker(
    input rawclk,
    input [3:0] statevalue,
    output reg speaker
```

```
);
    wire [31:0] clkarray0;
    wire [31:0] clkarray1;
    wire [31:0] clkarray2;
    wire [31:0] clkarray3;
    wire [31:0] clkarray4;
    wire [31:0] clkarray5;

    // Fixed sensitivity list to avoid simulator lockups
    always @(*) begin
        case(statevalue)
            4'd0:    speaker = clkarray0[15]; // Dflat
            4'd1:    speaker = clkarray1[15]; // E
            4'd2:    speaker = clkarray2[15]; // Gflat
            4'd3:    speaker = clkarray3[15]; // G
            4'd4:    speaker = clkarray4[15]; // Aflat
            4'd5:    speaker = clkarray5[15]; // B
            4'd6:    speaker = clkarray0[14]; // Dflat Octave Up
            4'd7:    speaker = clkarray1[14]; // E Octave Up
            4'd8:    speaker = clkarray2[14]; // Gflat Octave Up
            default: speaker = 1'b0;          // Output Off
        endcase
    end

    clockschemearray clkarrayinstance1(rawclk, clkarray0, clkarray1, clkarray2, clkarray3,
clkarray4, clkarray5);

endmodule

/* ############################################################################## */
/* Binary Upcounter Module */
module upcounter32(
    input ww,
    output reg [31:0] Qq
);
    // Safe initialization value for hardware registers
    initial Qq = 32'b0;

    always @(posedge ww) begin
        Qq <= Qq + 1'b1;
    end
endmodule

/* ############################################################################## */
/* Keyboard Priority Encoder Module */
module keyboard(
    input [7:0] key,
    output reg [3:0] out
);
    always @(*) begin
        casex(key)
            8'b1xxxxxxx: out = 4'b0000; // Dflat
            8'b01xxxxxx: out = 4'b0001; // E
            8'b001xxxxx: out = 4'b0010; // G flat
            8'b0001xxxx: out = 4'b0011; // G
            8'b00001xxx: out = 4'b0100; // A flat
            8'b000001xx: out = 4'b0101; // B
            8'b0000001x: out = 4'b0110; // Dflat up
            8'b00000001: out = 4'b0111; // E up
            default    : out = 4'b1111; // No keys pressed
        endcase
    end
endmodule

/* ############################################################################## */
/* Seven Segment Display Driver Module */
module sevsegdisplaynote(
    output reg [6:0] out1, // Note letter
    output reg [6:0] out2, // Accidental (b)
    input [3:0] note
);
```

```
    always @(*) begin
        case(note)
            4'b0000: begin out1 = 7'b0000001; out2 = 7'b1100000; end // Dflat
            4'b0001: begin out1 = 7'b0110000; out2 = 7'b1111111; end // E
            4'b0010: begin out1 = 7'b0100001; out2 = 7'b1100000; end // G flat
            4'b0011: begin out1 = 7'b0100001; out2 = 7'b1111111; end // G
            4'b0100: begin out1 = 7'b0001000; out2 = 7'b1100000; end // A flat
            4'b0101: begin out1 = 7'b0000000; out2 = 7'b1111111; end // B
            4'b0110: begin out1 = 7'b0000001; out2 = 7'b1100000; end // Dflat up
            4'b0111: begin out1 = 7'b0110000; out2 = 7'b1111111; end // E up
            default: begin out1 = 7'b1111111; out2 = 7'b1111111; end // Off
        endcase
    end
endmodule
```

4. Verilog Testbench Code

The testbench (tb_projectmaint.v) generates a 50 MHz clock (20 ns period), applies each of the eight keys in turn (active-low) followed by a release of all keys, and prints the key state, speaker output and both seven-segment outputs using $monitor.

```
`timescale 1ns/1ps

module tb_projectmaint;

    // Inputs
    reg rawclk;
    reg [7:0] keys;

    // Outputs
    wire speaker;
    wire [6:0] hex1;
    wire [6:0] hex2;

    // Instantiate the Unit Under Test (UUT)
    projectmaint uut (
        .rawclk(rawclk),
        .keys(keys),
        .speaker(speaker),
        .hex1(hex1),
        .hex2(hex2)
    );

    // Generate 50MHz clock (20ns period)
    initial begin
        rawclk = 0;
        forever #10 rawclk = ~rawclk;
    end

    // Stimulus and Monitoring
    initial begin
        // Default state: all keys released (active-HIGH input, so 8'hFF)
        keys = 8'hFF;

        // Print header
        $display("=================================================================");
        $display(" Time (ns) | Keys (Active-Low) | Speaker | Hex1    | Hex2    | Note");
        $display("=================================================================");

        // Continuous monitoring using text output with a music note symbol
        $monitor("%0t ns    | %b          | %b       | %b | %b | ♫",
                 $time, keys, speaker, hex1, hex2);

        // Cycle through all 8 keys one by one (active-low pulls individual bits to 0)
        #100 keys = 8'b01111111; // Tests Key 7 (State 0: Dflat)
```

```
        #200 keys = 8'b10111111; // Tests Key 6 (State 1: E)
        #200 keys = 8'b11011111; // Tests Key 5 (State 2: Gflat)
        #200 keys = 8'b11101111; // Tests Key 4 (State 3: G)
        #200 keys = 8'b11110111; // Tests Key 3 (State 4: Aflat)
        #200 keys = 8'b11111011; // Tests Key 2 (State 5: B)
        #200 keys = 8'b11111101; // Tests Key 1 (State 6: Dflat Up)
        #200 keys = 8'b11111110; // Tests Key 0 (State 7: E Up)
        #200 keys = 8'b11111111; // Release all keys (Default / Off)
        #200;

        $display("====================================================================");
        $display("Test completed successfully.");
        $finish;
    end

endmodule
```

5. Simulation

The design was simulated in ModelSim – Intel FPGA Starter Edition 2020.1 using the testbench above. Figure 3 shows the waveform window and the transcript output.

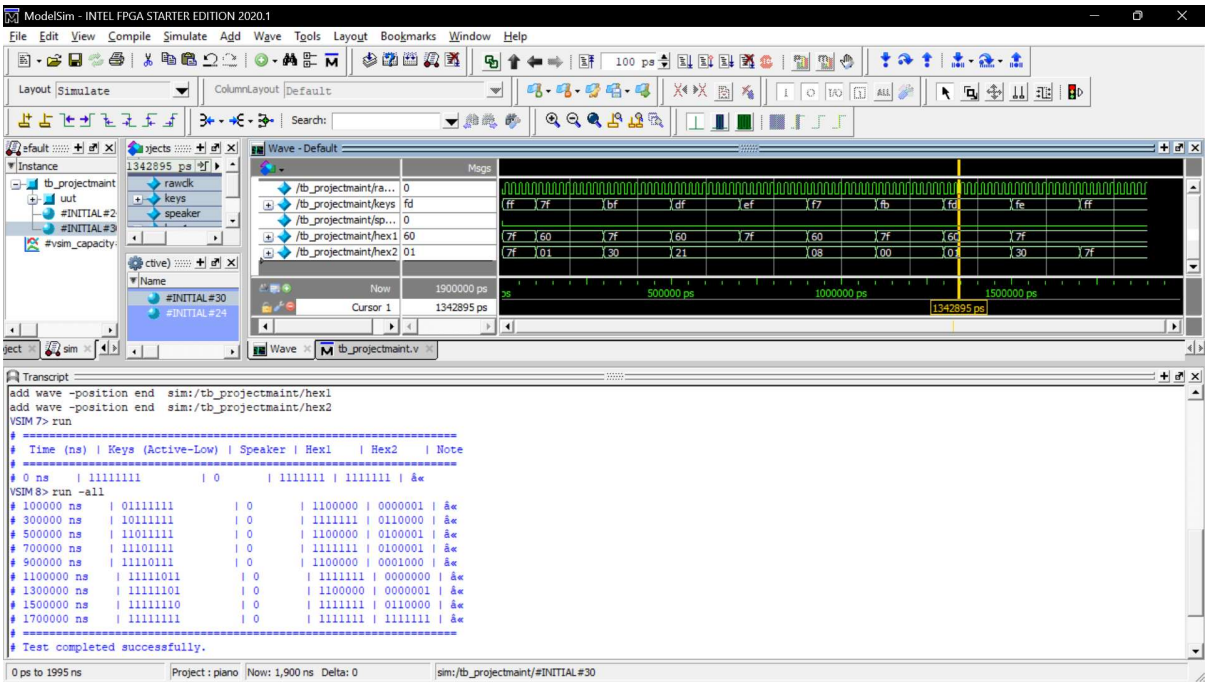


Figure 3: ModelSim waveform and transcript of tb_projectmaint

Observations from the simulation (key values are active-low):

Time (ns)	keys	hex1 (flat sign)	hex2 (note letter)
0	11111111	1111111	1111111
100	01111111	1100000	0000001
300	10111111	1111111	0110000
500	11011111	1100000	0100001
700	11101111	1111111	0100001
900	11110111	1100000	0001000
1100	11111011	1111111	0000000
1300	11111101	1100000	0000001
1500	11111110	1111111	0110000
1700	11111111	1111111	1111111

The seven-segment outputs change correctly for every key: keys that select a flat note (D♭, G♭, A♭) show the flat sign on hex1, natural notes leave hex1 blank, and releasing all keys turns both displays off. The transcript ends with “Test completed successfully.” at 1900 ns.

6. Pin Assignment / .tcl File

The keyboard inputs are taken from GPIO 5, 7, 9, 11, 13, 15, 17 and 19 of the JP5 40-pin header, and the speaker output is taken from a GPIO pin of the same header. Figure 4 shows the GPIO pin planner of the header connector.

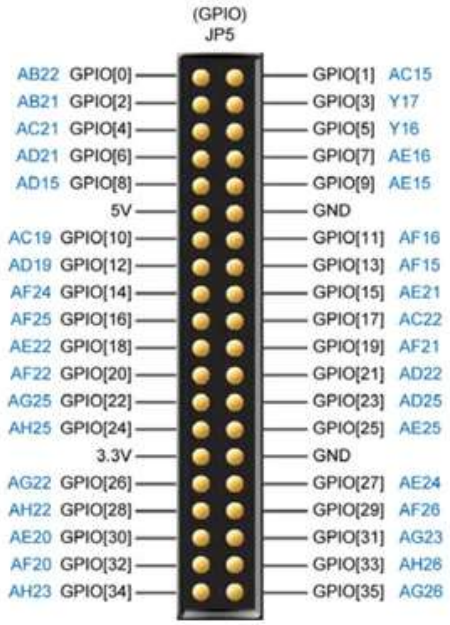


Figure 4: DE2-115 GPIO (JP5) header pin planner

6.1 Pin Assignment Table

Signal	FPGA Pin	Connected to	Description
rawclk	PIN_Y2	CLOCK_50	50 MHz clock
keys[7]	PIN_Y16	GPIO[5]	Key – D♭
keys[6]	PIN_AE16	GPIO[7]	Key – E
keys[5]	PIN_AE15	GPIO[9]	Key – G♭
keys[4]	PIN_AF16	GPIO[11]	Key – G
keys[3]	PIN_AF15	GPIO[13]	Key – A♭
keys[2]	PIN_AE21	GPIO[15]	Key – B
keys[1]	PIN_AC22	GPIO[17]	Key – D♭ (octave up)
keys[0]	PIN_AF21	GPIO[19]	Key – E (octave up)
speaker	PIN_AE25	GPIO[25]	Audio output
hex1[0..6]	G18, F22, E17, L26, L25, J22, H22	HEX0	Flat sign
hex2[0..6]	M24, Y22, W21, W22, W25, U23, U24	HEX1	Note letter

6.2 Tcl Script (piano.tcl)

```
# Pin assignments (Quartus Prime Tcl script)
# Usage:  quartus_sh -t pin_assignments.tcl
#   or in Quartus: Tools > Tcl Scripts... > select this file > Run
#
# Change PROJECT_NAME to match your .qpf project name.

load_package project

set PROJECT_NAME "your_project_name"

if {[is_project_open]} {
    set opened_here 0
} else {
    project_open $PROJECT_NAME
    set opened_here 1
}

# { node_name  pin_location }
set pin_list {
    {hex1[0]   PIN_G18}
    {hex1[1]   PIN_F22}
    {hex1[2]   PIN_E17}
    {hex1[3]   PIN_L26}
    {hex1[4]   PIN_L25}
    {hex1[5]   PIN_J22}
    {hex1[6]   PIN_H22}

    {hex2[0]   PIN_M24}
    {hex2[1]   PIN_Y22}
    {hex2[2]   PIN_W21}
    {hex2[3]   PIN_W22}
    {hex2[4]   PIN_W25}
    {hex2[5]   PIN_U23}
    {hex2[6]   PIN_U24}
```

```
    {keys[0]   PIN_AF21}
    {keys[1]   PIN_AC22}
    {keys[2]   PIN_AE21}
    {keys[3]   PIN_AF15}
    {keys[4]   PIN_AF16}
    {keys[5]   PIN_AE15}
    {keys[6]   PIN_AE16}
    {keys[7]   PIN_Y16}

    {rawclk    PIN_Y2}
    {speaker   PIN_AE25}
}

foreach entry $pin_list {
    set node [lindex $entry 0]
    set pin  [lindex $entry 1]

    # Location
    set_location_assignment $pin -to $node

    # I/O standard (2.5 V for all pins)
    set_instance_assignment -name IO_STANDARD "2.5 V" -to $node
}

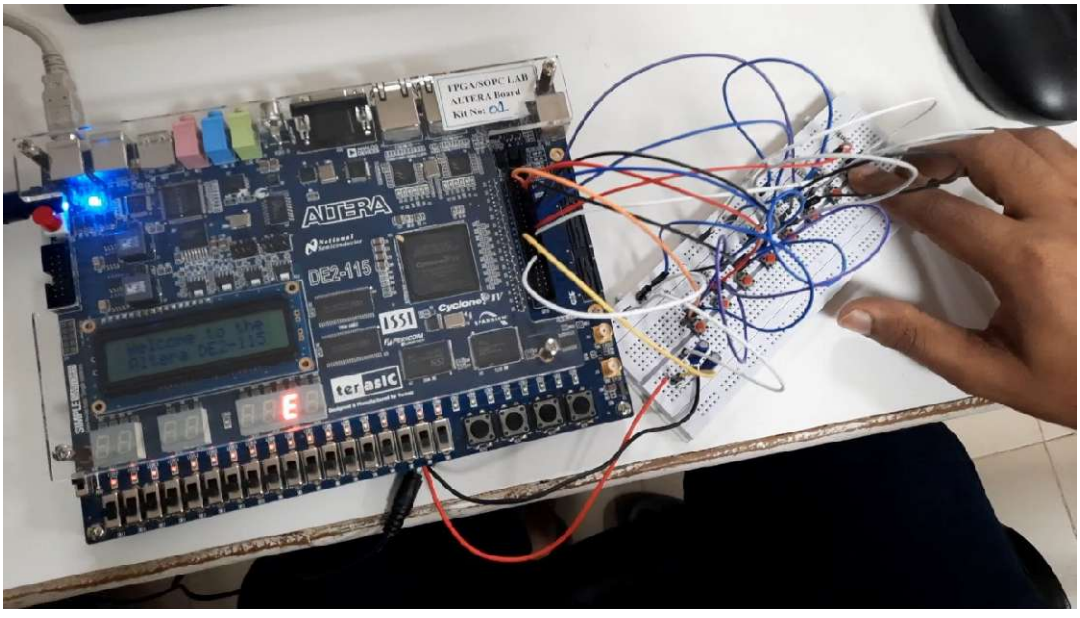
# Current strength / slew rate are left at their defaults (8mA, slew 2),
# so no explicit assignments are needed. To set them explicitly, uncomment:
# foreach entry $pin_list {
#     set node [lindex $entry 0]
#     set_instance_assignment -name CURRENT_STRENGTH_NEW "8MA" -to $node
# }

export_assignments

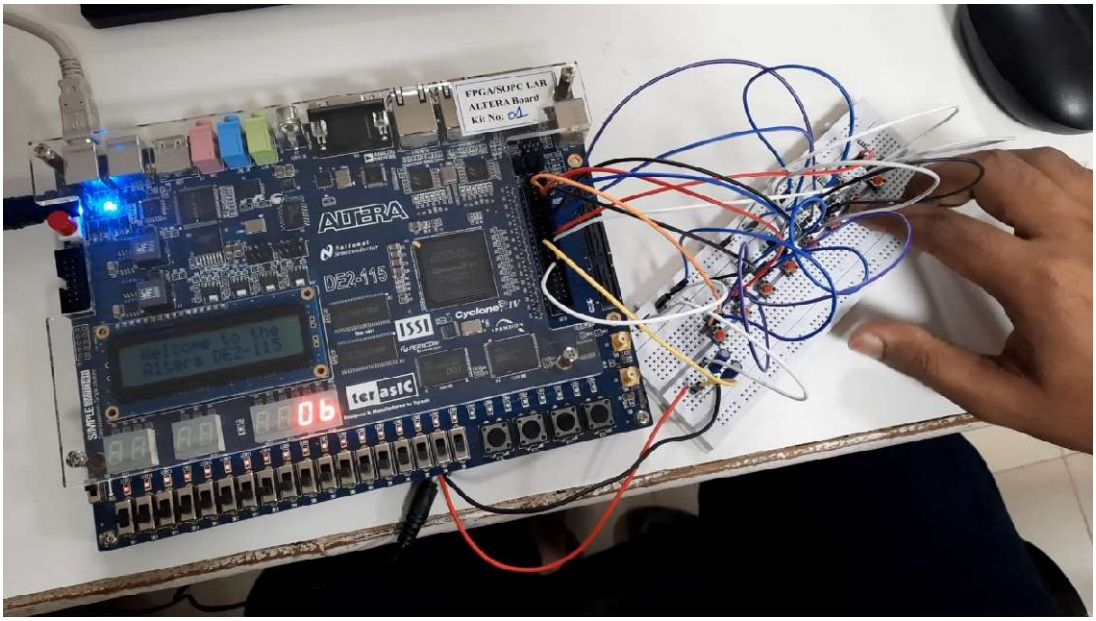
if {$opened_here} {
    project_close
}

puts "Pin assignments applied successfully."
```

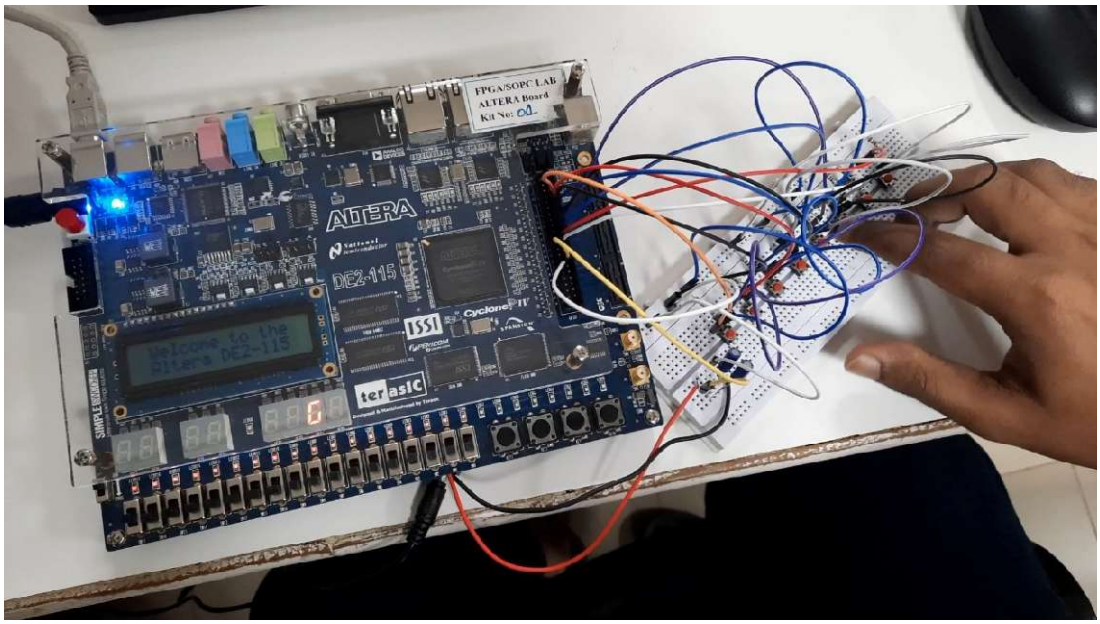
7. Output / Result



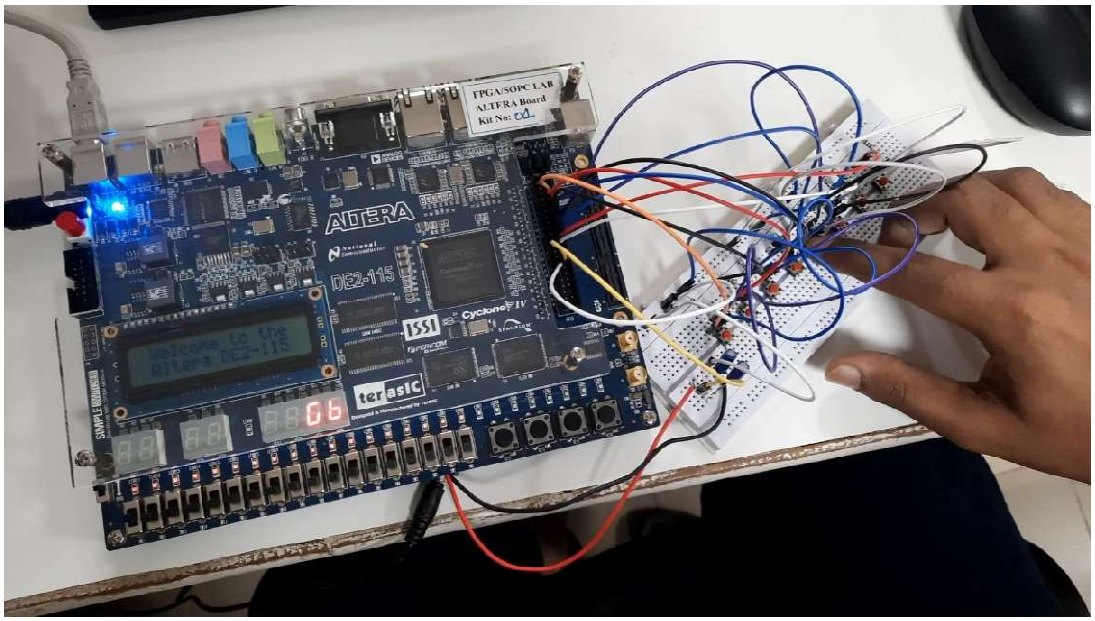
Key-1 input ; E output



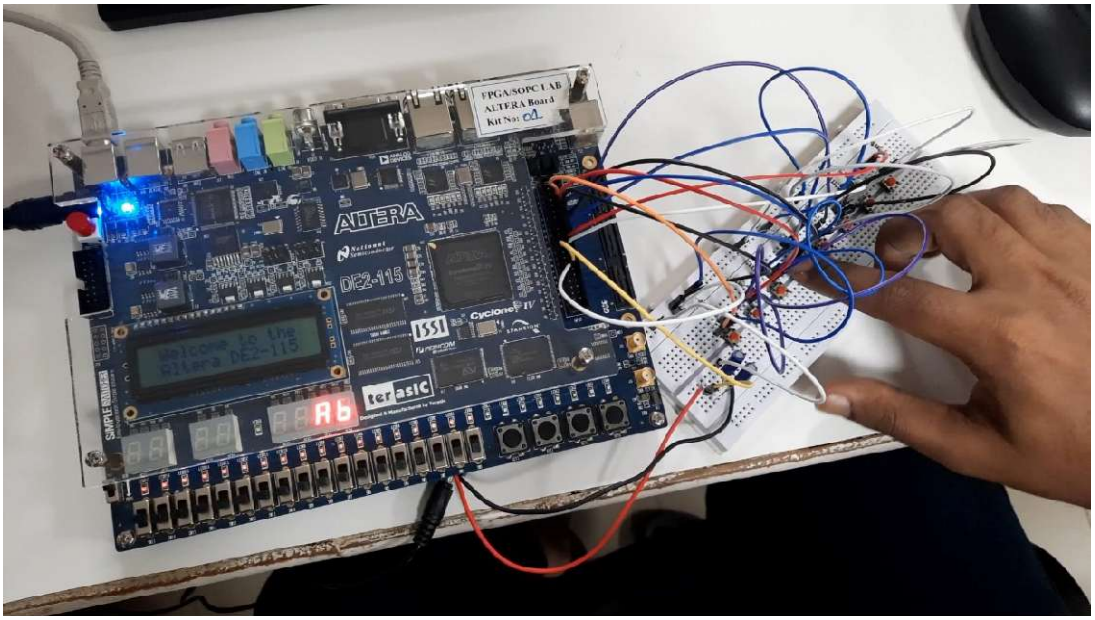
Key-2 input ; Ob output



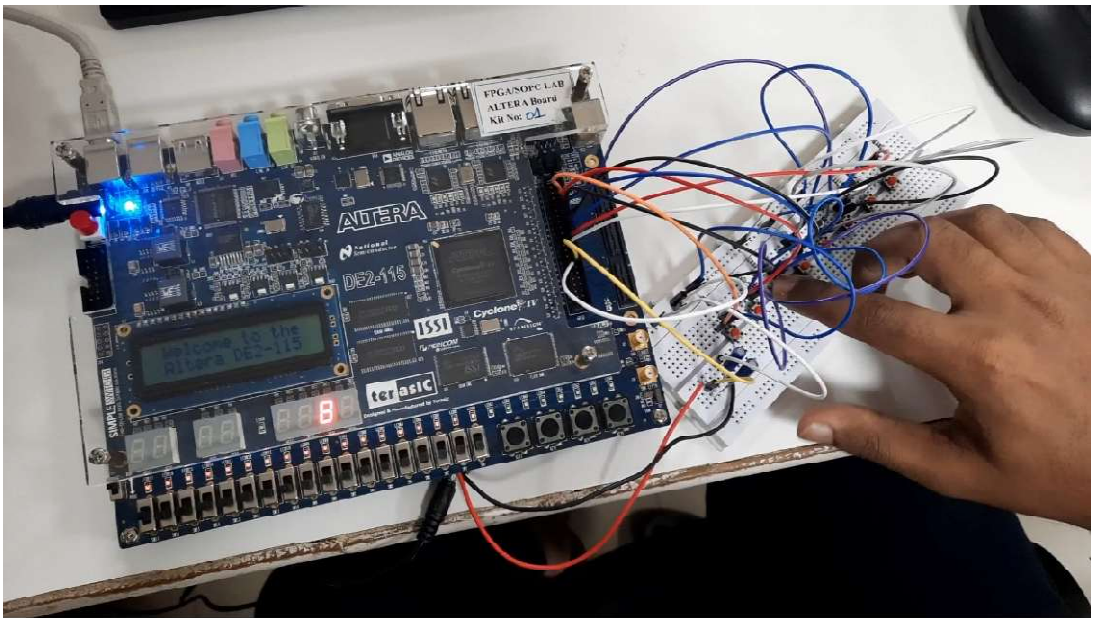
Key-3 input ; G output



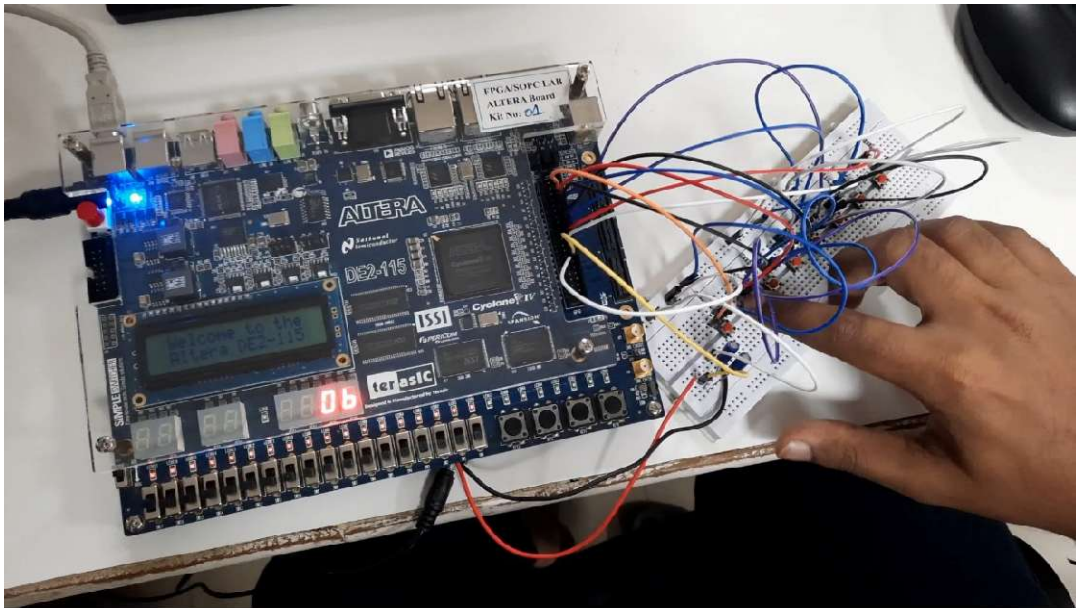
Key - 4 input ; Gb output



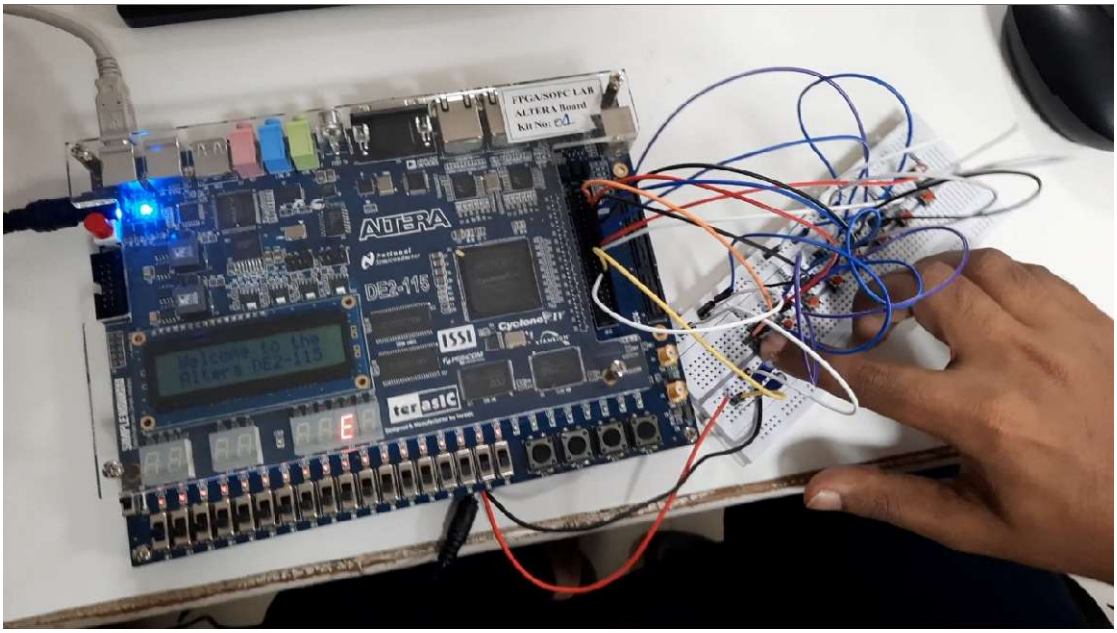
Key – 5 input ; Ab output



Key – 6 input ; B output



Key – 7 input ; Ob output



Key – 8 input ; E output

8. GitHub Repository & Demo Link

https://github.com/pulavarti-yogesh-vit/blues-piano-de2-115

https://drive.google.com/drive/folders/1vxEBrY7V0wm1QcRzNHjBwuxexZZF2SRm?usp=sharing